Code-Layout, Readability and Reusability

Standards, patterns, and principles that keep the codebase clean and maintainable

Project Name	LaunchPad AI
Maximum Marks	5 Marks

Well-structured code is essential for maintainability and collaboration. This table documents the standards, patterns, and principles LaunchPad AI followed to ensure the codebase is clean, readable, and highly reusable, with specific examples of how each practice was implemented.

Code Quality & Structure Details

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
1	**Folder/Directory Structure**	AI logic, views, and static assets are separated into their own directories rather than living in one file	/services (AI logic), /templates (views), /static (CSS & JS)
2	**Naming Conventions**	snake_case for functions and variables, PascalCase for database model classes, UPPER_CASE for constants	StartupReport class, generate_startup_analysis(), MAX_INPUT_LENGTH
3	**Code Readability & Comments**	Docstrings on every function explaining purpose and parameters; section-divider comments group related routes	Docstring in generate_startup_analysis(); section banners in app.py
4	**Modularity & Reusability**	AI logic is fully isolated in ai_engine.py so it can be reused or swapped independently of the Flask routes (DRY principle)	AIEngineError custom exception, _build_prompt() and _extract_json() helpers
5	**Formatting & Linting Tools**	use black for automated code formation and flask to enforce Pep 8 style guidelines and identify potential error	black and flake 8 configured and requirements.txt
6	**Error Handling Patterns**	Centralized custom exception (AIEngineError) for AI failures, plus Flask error handlers for 404/500 and try/except around all DB writes	AIEngineError, @app.errorhandler(500), db.session.rollback() on failure